TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TẬP TRÍ TUỆ NHÂN TẠO NÂNG CAO

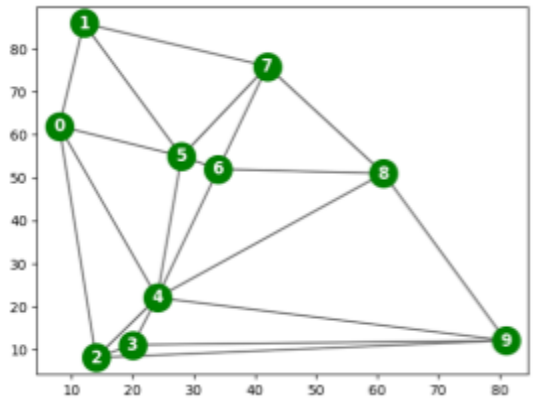
Sinh viên thực hiện:

Trần Ngọc Ánh	3122410013
Trần Tấn Đạt	3122410078
H' Như Lưk	3122410226
Nguyễn Tuyết Nhi	3122410282

TP. Hồ Chí Minh, tháng 10 năm 2026

I. Bài tập thực hành

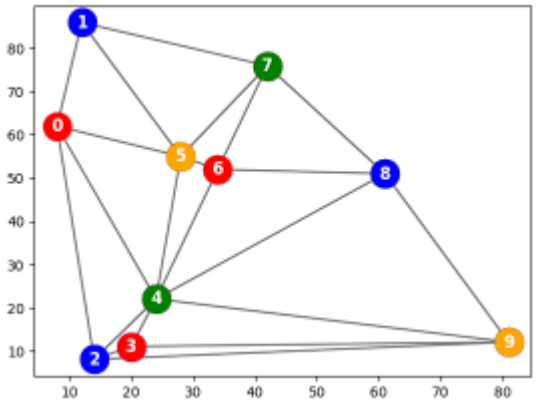
Bài 1:




```
[array([1, 2, 4, 5], dtype=int32),
 array([0, 5, 7], dtype=int32),
 array([0, 3, 4, 9], dtype=int32),
 array([2, 4, 9], dtype=int32),
 array([0, 2, 3, 5, 6, 8, 9], dtype=int32),
 array([0, 1, 4, 6, 7], dtype=int32),
 array([4, 5, 7, 8], dtype=int32),
 array([1, 5, 6, 8], dtype=int32),
 array([4, 6, 7, 9], dtype=int32),
 array([2, 3, 4, 8], dtype=int32)]
```

Đồ thị 10 đỉnh và mảng cấu trúc neighbors lần lượt từ đỉnh 0 đến 9

- X (các đỉnh): 10 đỉnh đồ thị từ 0 đến n-1.
- D (Tập màu): Gán từ 3 đến 4 màu cho cách đỉnh (red, blue, green / orange).
- C (Hai đỉnh kề nhau phải khác màu): Bất kỳ hai đỉnh i và j nào kề nhau thì màu khác nhau $color(i) \neq color(j)$.
- Điều kiện i < j: Vì đây là đồ thị vô hướng, cạnh nối đỉnh i với đỉnh j hoàn toàn giống cạnh nối đỉnh j với đỉnh i. Việc chỉ duyệt tạo ràng buộc khi $i < j$ giúp hệ thống không bị lưu trữ và xét lại các ràng buộc trùng lặp (ví dụ: chỉ tạo ràng buộc (0, 1) và bỏ qua (1, 0)), từ đó tối ưu bộ nhớ và tốc độ xử lý của thuật toán.
- Khảo sát tập màu:
  - Khảo sát 3 màu: Thuật toán không tìm thấy lời giải với số lượng node kiểm tra là 70.
  - Khảo sát 4 màu: Thuật toán tìm ra lời giải với số lượt node là 11:




```
Checked nodes: 11
CPU times: user 391 µs, sys: 0 ns, total: 391 µs
Wall time: 383 µs
{'0': 'red',
 '1': 'blue',
 '2': 'blue',
 '3': 'red',
 '4': 'green',
 '5': 'orange',
 '6': 'red',
 '7': 'green',
 '8': 'blue',
 '9': 'orange'}
```

Kết quả gán 4 màu cho đồ thị

- Phép gán đầy đủ và phép gán hợp lệ:

- Phép gán đầy đủ: Là khi mọi biến trong bài toán đều đã được gán một giá trị (toàn bộ 10 đỉnh đều đã được tô màu), bất kể việc gán đó có vi phạm ràng buộc hay không.
- Phép gán hợp lệ: Là khi các giá trị đã gán không vi phạm bất kỳ ràng buộc nào (chưa có 2 đỉnh kề nhau nào bị trùng màu). Một phép gán hợp lệ có thể chỉ là phép gán một phần (ví dụ mới tô màu 3 đỉnh hợp lệ). Lời giải cuối cùng của bài toán là một phép gán phải thỏa mãn cả hai yếu tố: vừa hợp lệ vừa đầy đủ.

Bài 2:

- Chạy complete(assignment, csp) và consistent(assignment, csp) với phép gán rỗng, một phần, đầy đủ và phép gán có hai đỉnh kề cùng màu.

```python
#Phép gán rỗng
print("Gán rỗng")
print("Complete:", complete({}, csp))
print("Consistent:", consistent({}, csp))
print("")

# Phép gán một phần hợp lệ
assignment = {'0': 'red', '2': 'blue'}
print("Gán assignment = {'0': 'red', '2': 'blue'}")
print("Complete:", complete(assignment, csp))
print("Consistent:", consistent(assignment, csp))
print("")

# Phép gán đầy đủ (10 đỉnh)
print("Gán assignment = {'0': 'red', '1': 'red', '2': 'red', '3': 'red', '4':'red',\n'5': 'red', '6': 'red', '7': 'red', '8': 'red', '9': 'red'}")
assignment = {'0': 'red', '1': 'red', '2': 'red', '3': 'red', '4': 'red', '5': 'red', '6': 'red', '7': 'red', '8': 'red', '9': 'red'}
print("Complete:", complete(assignment, csp))
print("Consistent:", consistent(assignment, csp))
print("")

# Phép gán vi phạm (hai đỉnh kề 0 và 1 cùng màu)
print("Gán assignment = {'0': 'red', '1': 'red', '2': 'blue'}")
assign_inconsistent = {'0': 'red', '1': 'red', '2': 'blue'}
print("Complete:", complete(assign_inconsistent, csp))
print("Consistent:", consistent(assign_inconsistent, csp))
```

```
Gán rỗng
Complete: False
Consistent: True

Gán assignment = {'0': 'red', '2': 'blue'}
Complete: False
Consistent: True

Gán assignment = {'0': 'red', '1': 'red', '2': 'red', '3': 'red', '4':'red',
'5': 'red', '6': 'red', '7': 'red', '8': 'red', '9': 'red'}
Complete: True
Consistent: False

Gán assignment = {'0': 'red', '1': 'red', '2': 'blue'}
Complete: False
Consistent: False
```

- assignment = {}: complete() = False vì chưa có đỉnh nào được gán màu và consistent() = True vì tập rỗng không vi phạm bất kỳ ràng buộc nào.
- assignment = {'0': 'red', '2': 'blue'}: complete() = False vì chỉ có 2 đỉnh được tô màu và consistent() = True vì đỉnh số 0 và đỉnh số 2 (hai đỉnh kề nhau) đã được gán hai màu khác nhau. .

- assignment = {'0': 'red', '1': 'red', '2': 'red', '3': 'red', '4': 'red', '5': 'red', '6': 'red', '7': 'red', '8': 'red', '9': 'red'}: complete() = True vì tất cả các đỉnh đều đã có một giá trị màu và consistent() = False vì tất cả các đỉnh đều được gán 'red' vi phạm ràng buộc.
- assignment = {'0': 'red', '1': 'red', '2': 'blue'}: complete() = False vì chỉ có 3 đỉnh được tô màu và consistent() = False vì i vì đỉnh 0 và đỉnh 1 vi phạm ràng buộc hai đỉnh kề nhau được gán 'red'.

- Kiểm thử:

- Phép gán thiếu nhưng hợp lệ: assignment = {'0': 'red'}, Kết quả mong đợi: complete() trả về False, consistent() trả về True.
- Phép gán đầy đủ nhưng vi phạm ràng buộc: assignment = {'0': 'red', '1': 'red', '2': 'red', '3': 'red', '4': 'red', '5': 'red', '6': 'red', '7': 'red', '8': 'red', '9': 'red'}. Kết quả mong đợi: complete() trả về True, consistent() trả về False.
- Phép gán đầy đủ và hợp lệ: assignment = {'0': 'red', '1': 'blue', '2': 'blue', '3': 'red', '4': 'green', '5': 'orange', '6': 'red', '7': 'green', '8': 'blue', '9': 'orange'}. Kết quả mong đợi: complete() trả về True, consistent() trả về True.

```python
#Phép gán thiếu nhưng hợp lệ
assignment = {'0': 'red'}
print("Complete:", complete({}, csp))
print("Consistent:", consistent({}, csp))
print("")

#Phép gán đầy đủ nhưng vi phạm ràng buộc
assignment = {'0': 'red', '1': 'red', '2': 'red', '3': 'red', '4': 'red', '5': 'red', '6': 'red', '7': 'red', '8': 'red', '9': 'red'}
print("Complete:", complete(assignment, csp))
print("Consistent:", consistent(assignment, csp))
print("")

#Phép gán đầy đủ và hợp lệ
assignment = {'0': 'red', '1': 'blue', '2': 'blue', '3': 'red', '4': 'green', '5': 'orange', '6': 'red', '7': 'green', '8': 'blue', '9': 'orange'}
print("Complete:", complete(assignment, csp))
print("Consistent:", consistent(assignment, csp))
```

```
Complete: False
Consistent: True

Complete: True
Consistent: False

Complete: True
Consistent: True
```

- Hai khái niệm complete() và consistent() đánh giá hai khía cạnh hoàn toàn độc lập của một phép gán trong bài toán CSP:

- complete() chỉ đánh giá mức độ hoàn thiện: Hàm này kiểm tra xem tất cả các biến (10 đỉnh) đã được gán giá trị (màu) hay chưa, hoàn toàn không quan tâm đến việc các màu đó có bị trùng lặp ở các cạnh kề nhau hay không.
- consistent() chỉ đánh giá tính hợp lệ: Hàm này rà soát tập ràng buộc để đảm bảo các giá trị đã được gán không vi phạm luật (hai đỉnh kề nhau phải khác màu), hoàn toàn không quan tâm đến việc còn bao nhiêu đỉnh chưa được tô màu.
  Chính vì sự độc lập này, một phép gán có thể đã được tô màu toàn bộ nhưng sai luật, hoặc đúng luật nhưng mới chỉ tô được vài đỉnh. Mục tiêu cuối cùng của thuật toán tìm kiếm CSP là tìm ra một trạng thái thỏa mãn đồng thời cả hai điều kiện này.

- Bài toán csp được lưu trữ dưới dạng một từ điển (dictionary) chứa 3 khóa (keys) chính:

- variables: Một danh sách (list) chứa tên của các biến cần tìm giá trị. Trong bài toán này, nó là danh sách các chuỗi đại diện cho các đỉnh, ví dụ: ['0', '1', '2', ..., '9'].

- domains: Một từ điển (dictionary) ánh xạ mỗi biến (đỉnh) tới tập miền giá trị hợp lệ của nó. Cụ thể là ánh xạ tới danh sách các màu, ví dụ: {'0': ['red', 'blue', 'green', 'orange'], '1': ['red', 'blue', 'green', 'orange'], ...}.
- constraints: Tập hợp các ràng buộc nhị phân giữa các biến, được biểu diễn dưới dạng danh sách các tuple (cặp đỉnh). Mỗi tuple (i, j) thể hiện rằng đỉnh *i* và đỉnh *j* kề nhau nên không được phép có cùng màu, ví dụ: ('0', '1'), ('0', '2').

- Phân biệt phép gán từng phần và nghiệm CSP: phép gán (assignment) là một từ điển lưu trữ biến nào đã được gán màu gì.

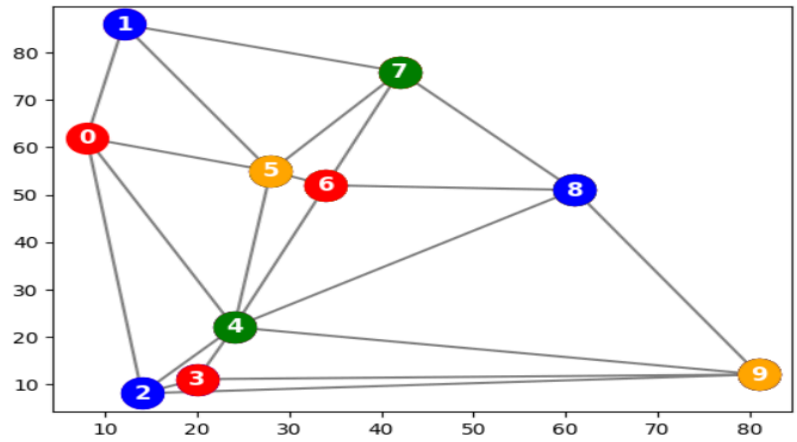
- Phép gán từng phần (Partial assignment): Là trạng thái mà quá trình tìm kiếm đang diễn ra, mới chỉ có một số biến (một vài đỉnh) được gán giá trị màu sắc, trong khi các biến khác vẫn đang để trống. Phép gán này có thể hợp lệ (không vi phạm luật consistent) nhưng nó chắc chắn không đầy đủ (complete = False). Ví dụ: {'0': 'red', '1': 'blue'}.
- Nghiệm CSP (CSP Solution): Là lời giải cuối cùng của bài toán. Đây là một phép gán thỏa mãn bắt buộc đồng thời hai điều kiện: đầy đủ (complete) tức là mọi đỉnh đều đã được tô màu, và hợp lệ (consistent) tức là không có bất kỳ cặp đỉnh kề nhau nào vi phạm luật cấm trùng màu.

Bài 3: Backtracking search và thứ tự chọn biến

Chạy backtrack_search(csp), theo dõi COUNT, thời gian chạy và kết quả tô màu. Trực quan hóa bằng đoạn mã vẽ trong notebook.

Sử dụng thuật toán Backtracking Search để giải bài toán tô màu đồ thị. Thuật toán lần lượt gán màu cho các đỉnh sao cho hai đỉnh có cạnh nối trực tiếp không được sử dụng cùng một màu. Khi một phép gán vi phạm ràng buộc, thuật toán quay lui để thử giá trị khác.

Trong quá trình chạy, theo dõi biến COUNT để ghi nhận số nút đệ quy đã xét, thời gian thực thi và kết quả tô màu đồ thị. Kết quả cho thấy thuật toán tìm được nghiệm hợp lệ cho đồ thị gồm 10 đỉnh với 4 màu.



Hình 3.1. Kết quả trực quan hóa đồ thị sau khi thực hiện Backtracking Search.

	Thuật toán	Có nghiệm	COUNT (nút đệ quy)	Thời gian (ms)
0	Backtracking gốc	True	11	0.208
1	MRV	True	11	0.474
2	MRV + LCV	True	11	1.023

Theo kết quả thực nghiệm, Backtracking gốc có COUNT = 11, thời gian chạy 0,208 ms và tìm được nghiệm. Đây là kết quả làm cơ sở để so sánh với các chiến lược chọn biến và thứ tự giá trị ở những phần tiếp theo.

- Phân tích select_unassigned_var(): phiên bản ban đầu chọn biến chưa gán đầu tiên; cài đặt Minimum Remaining Values (MRV) trên miền còn hợp lệ.

Ở phiên bản ban đầu, hàm select_unassigned_var() chọn biến chưa được gán đầu tiên theo thứ tự trong danh sách biến. Cách chọn này đơn giản và ít tốn chi phí tính toán, tuy nhiên chưa tận dụng thông tin về số giá trị hợp lệ còn lại của từng biến.

Để cải thiện chiến lược chọn biến, cần cài đặt Minimum Remaining Values (MRV). Chiến lược này ưu tiên chọn biến chưa gán có miền giá trị hợp lệ nhỏ nhất. Việc chọn biến bị hạn chế nhiều nhất trước có thể giúp phát hiện sớm các nhánh không có nghiệm, từ đó giảm lượng tìm kiếm trong một số bài toán.

Trong lần chạy thực nghiệm này, MRV cũng tìm được nghiệm hợp lệ với COUNT = 11 và thời gian chạy 0,474 ms. So với Backtracking gốc, số nút đệ quy không giảm, trong khi thời gian chạy tăng do phải tính toán miền giá trị hợp lệ để lựa chọn biến.

```
            # MRV: ưu tiên biến có ít màu hợp lệ nhất; phá hòa theo thứ tự gốc.
            return min(unassigned, key=lambda v: len(legal_values(v, assignment, csp)))
        raise ValueError("strategy phải là 'original' hoặc 'mrv'")

    def order_values(var, assignment, csp, strategy="original"):
        values = list(csp["domains"][var])
        if strategy == "original":
            return values
        if strategy == "lcv":
```

- Thử bổ sung một chiến lược thứ tự giá trị, ví dụ Least Constraining Value (LCV); so sánh kết quả với phiên bản gốc trên cùng đồ thị.

Bổ sung chiến lược Least Constraining Value (LCV) để sắp xếp các màu có thể gán cho biến. LCV ưu tiên màu gây ít hạn chế nhất đối với miền giá trị của các biến chưa được gán ở những đỉnh lân cận. Mục đích là giữ lại nhiều lựa chọn cho các biến tiếp theo, qua đó tăng khả năng tìm được nghiệm.

Kết quả thực nghiệm trên cùng đồ thị được trình bày trong Bảng 3.1.

Thuật toán	Có nghiệm	COUNT	Thời gian (ms)
Backtracking gốc	Có	11	0,208
MRV	Có	11	0,474

MRV + LCV	Có	11	1,023

Nhận xét: Cả ba phương pháp đều tìm được nghiệm hợp lệ và có cùng COUNT = 11. MRV và MRV + LCV chưa làm giảm số nút đệ quy trên đồ thị đang thử. Thời gian chạy tăng từ 0,208 ms lên 0,474 ms và 1,023 ms, cho thấy chi phí tính toán bổ sung của các chiến lược heuristic có thể lớn hơn lợi ích thu được khi đồ thị nhỏ.

=> Như vậy, MRV và LCV có thể giúp cải thiện quá trình tìm kiếm ở những bài toán phù hợp, nhưng không đảm bảo luôn giảm thời gian chạy hoặc số nút được xét trong mọi trường hợp.

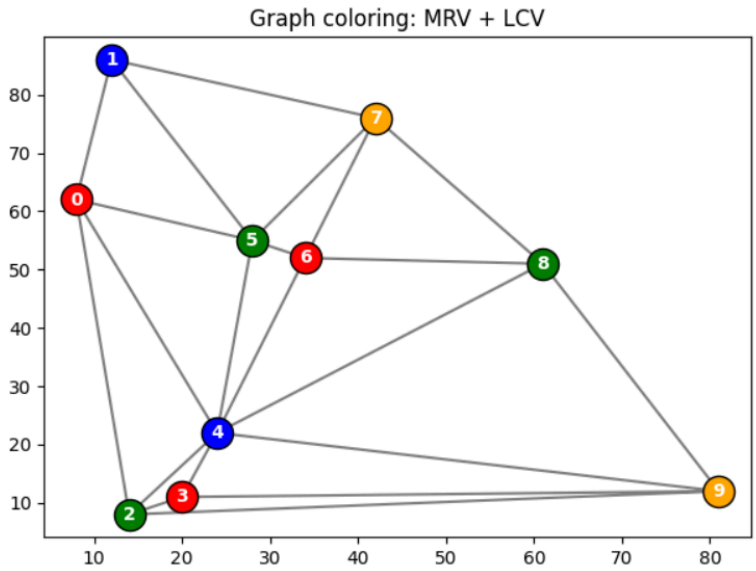
```
0 giây
        # LCV: màu loại bỏ ít lựa chọn hợp lệ nhất khỏi các láng giềng chưa gán.
        def impact(value):
            trial = dict(assignment)
            trial[var] = value
            removed = 0
            for a, b in csp["constraints"]:
                neighbor = b if a == var else (a if b == var else None)
                if neighbor is not None and neighbor not in trial:
                    if value in legal_values(neighbor, trial, csp):
                        removed += 1
            return removed
        return sorted(values, key=impact)
    raise ValueError("strategy phải là 'original' hoặc 'lcv'")

def backtracking_variant(csp, var_strategy="original", value_strategy="original"):
    """Backtracking không Forward Checking; trả về (nghiệm, COUNT, thời gian)."""
    count = 0
    start = time.perf_counter()

    def recurse(assignment):
        nonlocal count
        count += 1
        if len(assignment) == len(csp["variables"]):
            return dict(assignment)
        var = select_var_order(assignment, csp, var_strategy)
        for value in order_values(var, assignment, csp, value_strategy):
            assignment[var] = value
            if consistent(assignment, csp):
                result = recurse(assignment)
                if result is not None:
                    return result
            del assignment[var]
        return None

    result = recurse({})
    elapsed = time.perf_counter() - start
    return result, count, elapsed
```

Phần cài đặt hoặc sắp xếp giá trị theo LCV



Đồ thị tô màu bằng MRV + LCV

Bài 4. Forward checking và Min-Conflicts

- Bổ sung Forward Checking: sau khi gán màu cho một đỉnh, loại màu xung đột khỏi miền các đỉnh láng giềng chưa gán; hoàn tác miền khi quay lui.

Forward Checking là kỹ thuật cải tiến thuật toán Backtracking bằng cách kiểm tra các miền giá trị của những biến chưa được gán sau mỗi lần gán giá trị. Trong bài toán tô màu đồ thị, khi một đỉnh được gán màu, thuật toán loại màu đó khỏi miền giá trị của các đỉnh láng giềng chưa gán. Nếu miền giá trị của một đỉnh trở thành rỗng, phép gán hiện tại không thể dẫn đến nghiệm hợp lệ. Thuật toán sẽ quay lui và khôi phục các miền giá trị đã thay đổi trước khi thử phương án khác. Việc hoàn tác này giúp tránh ảnh hưởng đến các nhánh tìm kiếm khác.

So với Backtracking thông thường, Forward Checking có khả năng phát hiện sớm các nhánh không hợp lệ. Qua thực nghiệm, thuật toán tìm được nghiệm ở một số đồ thị với số bước và thời gian chạy thấp.

[12] 0 giây

```
def backtracking_forward_checking(csp, var_strategy="mrv", value_strategy="lcv"):
    """Trả về (nghiệm, COUNT, thời gian). Mỗi nhánh có bản sao miền để hoàn tác an toàn."""
    count = 0
    start = time.perf_counter()
    initial_domains = {v: list(csp["domains"][v]) for v in csp["variables"]}

    def recurse(assignment, current_domains):
        nonlocal count
        count += 1
        if len(assignment) == len(csp["variables"]):
            return dict(assignment)

        # Chọn biến dựa trên miền hiện còn sau Forward Checking.
        unassigned = [v for v in csp["variables"] if v not in assignment]
        if var_strategy == "mrv":
            var = min(unassigned, key=lambda v: len(current_domains[v]))
        else:
            var = unassigned[0]

        values = list(current_domains[var])
        if value_strategy == "lcv":
            def impact(value):
                removed = 0
                for a, b in csp["constraints"]:
                    nei = b if a == var else (a if b == var else None)
                    if nei is not None and nei not in assignment and value in current_domains[nei]:
                        removed += 1
                return removed
            values.sort(key=impact)

        for value in values:
            if any(
                (a == var and b in assignment and assignment[b] == value) or
                (b == var and a in assignment and assignment[a] == value)
                for a, b in csp["constraints"]
            ):
                continue

            new_assignment = dict(assignment)
            new_assignment[var] = value
            new_domains = {v: list(d) for v, d in current_domains.items()}
            failed = False

            for a, b in csp["constraints"]:
                nei = b if a == var else (a if b == var else None)
                if nei is not None and nei not in new_assignment:
                    new_domains[nei] = [x for x in new_domains[nei] if x != value]
                    if not new_domains[nei]:
                        failed = True
                        break

            if not failed:
                result = recurse(new_assignment, new_domains)
                if result is not None:
                    return result
        return None

    result = recurse({}, initial_domains)
    return result, count, time.perf_counter() - start
```

Hàm Forward Checking, bao gồm loại màu xung đột và khôi phục miền khi quay lui

	Thuật toán	Có nghiệm	COUNT / bước	Thời gian (ms)
0	Backtracking + Forward Checking	True	11	0.391
1	Min-Conflicts	True	14	1.616

Bảng 4.1 Kết quả chạy thử Forward Checking và Min-Conflicts ở một đồ thị

Trong kết quả thử nghiệm ban đầu, Forward Checking tìm được nghiệm với COUNT = 11, thời gian chạy 0,391 ms. Kết quả này cho thấy thuật toán hoạt động thành công trên trường hợp được thử nghiệm. Tuy nhiên, cần so sánh trên nhiều đồ thị để đánh giá khách quan hơn.

- Cài đặt tìm kiếm cục bộ Min-Conflicts theo yêu cầu Assignment: khởi tạo một phép gán đầy đủ, chọn biến đang vi phạm rồi đổi sang giá trị làm giảm số ràng buộc vi phạm.

Min-Conflicts là thuật toán tìm kiếm cục bộ giải bài toán ràng buộc bằng cách bắt đầu từ một phép gán đầy đủ cho tất cả các biến, sau đó liên tục điều chỉnh các biến đang vi phạm ràng buộc.

Trong bài toán tô màu đồ thị, thuật toán thực hiện các bước:

1. Khởi tạo màu cho tất cả các đỉnh.
2. Kiểm tra các cạnh để xác định những đỉnh đang tham gia vào xung đột.
3. Chọn một đỉnh đang vi phạm ràng buộc.
4. Chọn màu làm giảm số xung đột của đỉnh đó, ưu tiên màu tạo ra ít xung đột nhất.
5. Lặp lại cho đến khi không còn xung đột hoặc đạt giới hạn số bước.

Ưu điểm của Min-Conflicts là có thể tìm nghiệm mà không cần duyệt toàn bộ cây tìm kiếm. Tuy nhiên, thuật toán không đảm bảo tìm được nghiệm trong mọi lần chạy vì kết quả phụ thuộc vào phép gán ban đầu, cấu trúc đồ thị và giới hạn số bước.

```python
def min_conflicts(csp, max_steps=10000, seed=None):
    """Min-Conflicts; trả về (nghiệm hoặc None, số bước, thời gian)."""
    rng = np.random.default_rng(seed)
    variables = list(csp["variables"])
    domains = {v: list(csp["domains"][v]) for v in variables}
    neighbors_map = {v: set() for v in variables}
    for a, b in csp["constraints"]:
        neighbors_map[a].add(b)
        neighbors_map[b].add(a)

    start = time.perf_counter()
    assignment = {v: rng.choice(domains[v]) for v in variables}

    def conflicts_for(v, color):
        return sum(assignment[nei] == color for nei in neighbors_map[v])

    for step in range(max_steps + 1):
        conflicted = [v for v in variables if conflicts_for(v, assignment[v]) > 0]
        if not conflicted:
            return assignment, step, time.perf_counter() - start
        if step == max_steps:
            break
        var = rng.choice(conflicted)
        scores = {color: conflicts_for(var, color) for color in domains[var]}
        best_score = min(scores.values())
        best_colors = [color for color, score in scores.items() if score == best_score]
        assignment[var] = rng.choice(best_colors)

    return None, max_steps, time.perf_counter() - start
```

Hàm Min-Conflicts và phần kiểm tra số xung đột

Kết quả thí nghiệm cho thấy Min-Conflicts tìm được nghiệm ở nhiều trường hợp, nhưng với đồ thị 12 đỉnh và 3 màu, thuật toán đạt giới hạn 5.000 bước mà không tìm được nghiệm trong các lần chạy đã thực hiện.

- Với các giá trị n khác nhau và miền 3/4 màu, tạo nhiều đồ thị ngẫu nhiên; so sánh tỷ lệ tìm nghiệm, COUNT hoặc số bước, thời gian chạy. Lưu ý bài toán 3 màu có thể vô nghiệm.

Tiến hành thử nghiệm trên các đồ thị có số đỉnh $n = 8, 12, 16$, với miền giá trị gồm 3 màu hoặc 4 màu. Mỗi cấu hình được thử trên 5 đồ thị ngẫu nhiên. Hai thuật toán Forward Checking và Min-Conflicts được chạy trên cùng các trường hợp để so sánh tỷ lệ thành công, số nút hoặc số bước trung bình và thời gian thực thi trung bình.

Bảng 4.2. Kết quả tổng hợp thực nghiệm

	n	Màu	Thuật toán	Tỷ_lệ_thành_công	Cạnh_trung_bình	COUNT_bước_trung_bình	Thời_gian_ms_trung_bình
0	8	3	Forward Checking	60.0	15.8	14.2	0.27
1	8	3	Min-Conflicts	60.0	15.8	2005.8	86.77
2	8	4	Forward Checking	100.0	15.8	9.0	0.13
3	8	4	Min-Conflicts	100.0	15.8	5.6	0.41
4	12	3	Forward Checking	0.0	26.4	34.0	0.66
5	12	3	Min-Conflicts	0.0	26.4	5000.0	244.05
6	12	4	Forward Checking	100.0	27.2	13.0	0.38
7	12	4	Min-Conflicts	100.0	27.2	24.2	2.47
8	16	3	Forward Checking	20.0	38.4	24.6	0.62
9	16	3	Min-Conflicts	20.0	38.4	4058.6	240.41
10	16	4	Forward Checking	100.0	38.2	18.0	0.57
11	16	4	Min-Conflicts	100.0	38.2	26.2	1.76

Nhận xét kết quả

- Ảnh hưởng của số màu: Với 4 màu, cả hai thuật toán đều đạt tỷ lệ thành công 100% trong các trường hợp thử nghiệm. Với 3 màu, tỷ lệ thành công thấp hơn: 60% đối với đồ thị 8 đỉnh, 0% đối với đồ thị 12 đỉnh và 20% đối với đồ thị 16 đỉnh. Kết quả cho thấy việc tăng số màu giúp các đồ thị trong bộ thử nghiệm dễ tìm được nghiệm hơn.
- So sánh hai thuật toán: Forward Checking có thời gian chạy trung bình thấp hơn Min-Conflicts ở tất cả các cấu hình được thử nghiệm. Khi dùng 3 màu, Min-Conflicts cần trung bình 2.005,8 bước ở đồ thị 8 đỉnh và 4.058,6 bước ở đồ thị 16 đỉnh. Với đồ thị 12 đỉnh và 3 màu, thuật toán đạt giới hạn 5.000 bước mà không tìm được nghiệm trong cả 5 lần chạy.
- Forward Checking cũng không tìm được nghiệm ở một số cấu hình 3 màu, nhưng số nút tìm kiếm thấp và thời gian chạy ngắn. Với 4 màu, Forward Checking đạt tỷ lệ thành công 100% và có thời gian trung bình từ 0,13 đến 0,57 ms.

Kết luận: Qua thực nghiệm, Forward Checking phù hợp khi cần tìm kiếm có hệ thống và phát hiện sớm các nhánh không hợp lệ. Min-Conflicts có ưu điểm là tìm kiếm cục bộ trên phép gán đầy đủ, nhưng số bước và thời gian chạy có thể tăng đáng kể khi bài toán khó hoặc số màu không đủ để tìm nghiệm.

Trong bộ dữ liệu đã thử, 4 màu cho tỷ lệ thành công cao hơn 3 màu. Tuy nhiên, hiệu quả của mỗi thuật toán vẫn phụ thuộc vào cấu trúc đồ thị, cách khởi tạo và giới hạn tìm kiếm.

Bài 5:Biểu diễn bài toán n quân hậu

Ta biểu diễn một bàn cờ bằng một danh sách hoặc mảng board có n phần tử:

board = $[r_o, r_1, \dots, r_{n-1}]$

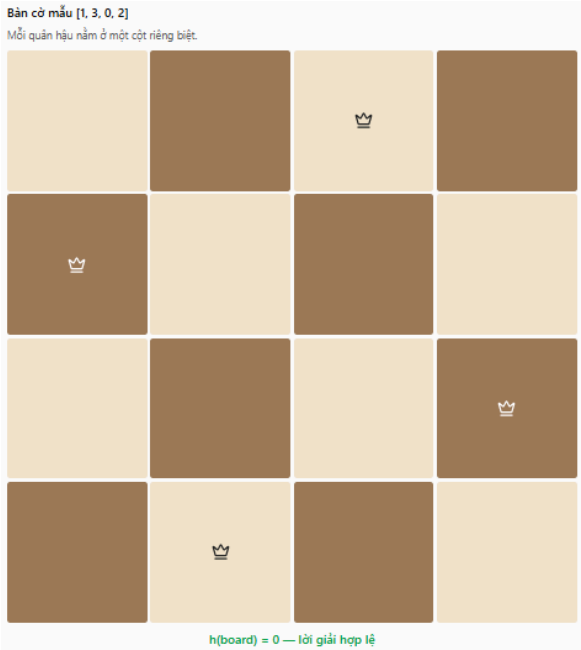
Trong đó:

- len(board) = n : bàn cờ có n cột và n quân hậu.
- board[i] là chỉ số hàng của quân hậu nằm tại cột i.

- Chỉ số hàng và cột được đánh số từ 0 đến n - 1.

- Mỗi phần tử có giá trị từ 0 đến n - 1.

Ví dụ, với bàn cờ: board = [1, 3, 0, 2]



Bốn quân hậu lần lượt nằm tại các vị trí (1, 0), (3, 1), (0, 2), (2, 3), theo thứ tự (hàng, cột). Bàn cờ này là một lời giải hợp lệ vì không có hai quân hậu nào nằm cùng hàng hoặc cùng đường chéo.

1. Khảo sát các hàm

1.1. Hàm random board(n)

Mã code:

```
def random_board(n):
    """Creates a random board of size n x n. Note that only a single queen is placed in each column!"""
    return(np.random.randint(0,n, size = n))
```

Chức năng: Tạo ngẫu nhiên một bàn cờ kích thước n x n.

Hàm trả về một mảng có n phần tử, trong đó mỗi phần tử là chỉ số hàng của quân hậu tại cột tương ứng. Các giá trị được chọn ngẫu nhiên từ 0 đến n - 1.

Kết quả thực tế có thể khác nhau tùy vào trạng thái sinh số ngẫu nhiên. Hàm này đảm bảo mỗi cột có đúng một quân hậu, nhưng không đảm bảo các quân hậu không tấn công nhau.

1.2. Hàm conflicts(board)

Mã code: def comb2(n): return n*(n-1)//2

Hàm comb2(n) tính số cặp có thể chọn từ n phần tử: $C_n^2 = \frac{n(n-1)}{2}$. Tiếp theo, hàm conflicts(board) sử dụng kết quả này để tính số cặp quân hậu xung đột.

Hàm conflicts(board) đếm số quân hậu nằm cùng hàng và cùng các đường chéo. Vì mỗi cột đã có đúng một quân hậu nên không cần đếm xung đột theo cột.

1.3. Hàm show_board(board)

Mã code: show_board(board)

Chức năng: Hiển thị bàn cờ dưới dạng hình ảnh, trong đó các ô được tô màu xen kẽ và quân hậu được biểu diễn bằng ký hiệu ♛.

Ngoài việc vẽ bàn cờ, hàm còn in ra số xung đột của trạng thái hiện tại.

2. Xây dựng hàm mục tiêu h(board)

Hàm mục tiêu được định nghĩa là số cặp quân hậu có thể tấn công nhau:

h(board) = số cặp quân hậu cùng hàng hoặc cùng đường chéo

Giả sử quân hậu thứ i nằm tại vị trí (board[i],i), quân hậu thứ j nằm tại vị trí (board[j],j).

Hai quân hậu tấn công nhau khi thỏa mãn ít nhất một trong hai điều kiện:

+ Cùng hàng: board[i] = board[j].

+ Cùng đường chéo: |board[i] - board[j]| = | i - j |. Do mỗi cột chỉ có một quân hậu nên ta chỉ cần kiểm tra các cặp quân hậu có chỉ số i < j.

Code hàm mục tiêu:

```
def h(board):
    n = len(board)
    conflicts = 0

    for i in range(n):
        for j in range(i + 1, n):
            # Hai quân hậu cùng hàng
            same_row = board[i] == board[j]

            # Hai quân hậu cùng đường chéo
            same_diagonal = abs(board[i] - board[j]) == abs(i - j)

            if same_row or same_diagonal:
                conflicts += 1

    return conflicts
```

3. Tạo bàn cờ với n=4 và n=8

3.1. Tạo bàn cờ 4 x 4

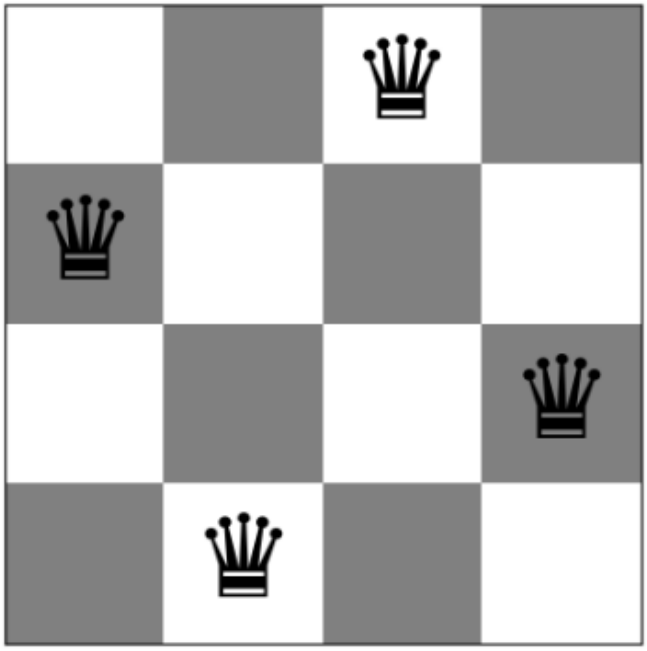
Mã code:

```
board4 = [1, 3, 0, 2]
show_board(board4)

print("Bàn cờ:", board4)
print("h(board) =", h(board4))
print("conflicts(board) =", conflicts(board4))
```

Kết quả:

```
Board with 0 conflicts.
```



```
Bàn cờ: [1, 3, 0, 2]
h(board) = 0
conflicts(board) = 0
```

Nhận xét: Bàn cờ có bốn quân hậu, mỗi cột có đúng một quân hậu. Không có hai quân hậu nào cùng hàng hoặc cùng đường chéo nên h(board) = 0. Đây là một lời giải hợp lệ cho bài toán 4 quân hậu.

3.2. Tạo bàn cờ 8 x 8

Đối với bàn cờ 8 x 8, ta sử dụng hàm random_board(8) để tạo một trạng thái ngẫu nhiên. Đồng thời, có thể sử dụng một trạng thái đã biết là lời giải để kiểm tra hàm mục tiêu.

Mã code:

```
# Tạo bàn cờ 8x8 ngẫu nhiên
board8_random = random_board(8)
show_board(board8_random)

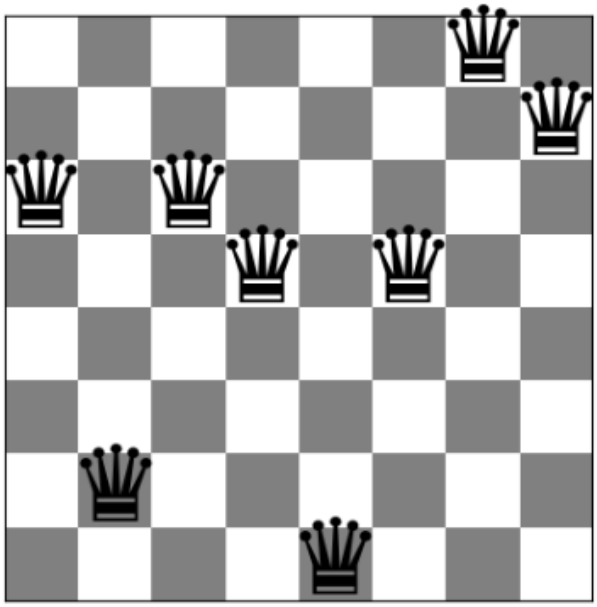
print("Bàn cờ ngẫu nhiên:", board8_random)
print("h(board) =", h(board8_random))
print("conflicts(board) =", conflicts(board8_random))


# Một lời giải hợp lệ cho bài toán 8 quân hậu
board8_solution = [0, 4, 7, 5, 2, 6, 1, 3]
show_board(board8_solution)

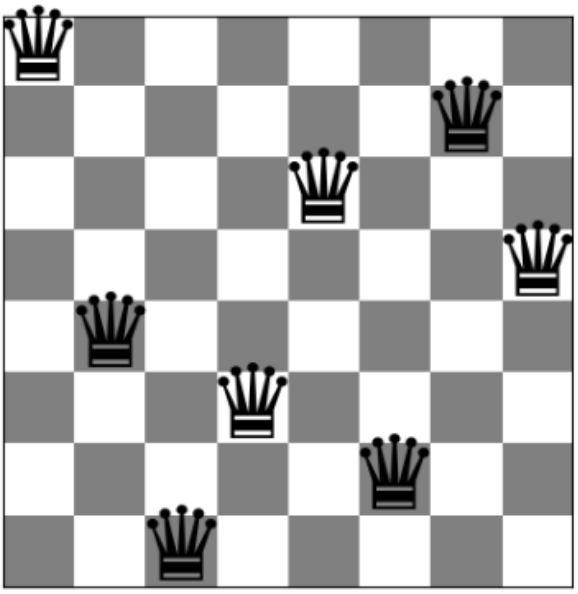
print("Bàn cờ lời giải:", board8_solution)
print("h(board) =", h(board8_solution))
print("conflicts(board) =", conflicts(board8_solution))
```

Kết quả:

```
Board with 6 conflicts.
```



```
Bàn cờ ngẫu nhiên: [2 6 2 3 7 3 0 1]
h(board) = 6
conflicts(board) = 6
Board with 0 conflicts.
```



```
Bàn cờ lời giải: [0, 4, 7, 5, 2, 6, 1, 3]
h(board) = 0
conflicts(board) = 0
```

Đối với bàn cờ ngẫu nhiên, giá trị h(board) có thể bằng 0 hoặc lớn hơn 0, tùy vào cách sắp xếp các quân hậu.

4. Kiểm tra điều kiện h(board) = 0

h(board) = 0 <=> Không có cặp quân hậu nào tấn công nhau

Điều này có nghĩa:

- Nếu h(board) = 0, không tồn tại cặp quân hậu nào cùng hàng hoặc cùng đường chéo.

- Nếu không có cặp quân hậu nào tấn công nhau, không có cặp nào được tính là xung đột nên h(board) = 0.

Để kiểm tra, ta viết thêm hàm xác định xem bàn cờ có lời giải hợp lệ hay không.

```
def is_solution(board):
    n = len(board)

    # Kiểm tra giá trị hàng có hợp lệ không
    if any(row < 0 or row >= n for row in board):
        return False

    # Kiểm tra từng cặp quân hậu
    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j]:
                return False

            if abs(board[i] - board[j]) == abs(i - j):
                return False

    return True

# Kiểm tra bàn cờ mẫu
boards = [
    [1, 3, 0, 2],
    [0, 4, 7, 5, 2, 6, 1, 3],
    random_board(4),
    random_board(8)
]

for board in boards:
    print("Bàn cờ:", board)
    print("h(board) =", h(board))
    print("Lời giải hợp lệ:", is_solution(board))
    print("Kiểm tra điều kiện:", (h(board) == 0) == is_solution(board))
    print("-" * 40)
```

Kết quả:

```
Bàn cờ: [1, 3, 0, 2]
h(board) = 0
Lời giải hợp lệ: True
Kiểm tra điều kiện: True
----------------------------------------
Bàn cờ: [0, 4, 7, 5, 2, 6, 1, 3]
h(board) = 0
Lời giải hợp lệ: True
Kiểm tra điều kiện: True
----------------------------------------
Bàn cờ: [3 1 3 1]
h(board) = 2
Lời giải hợp lệ: False
Kiểm tra điều kiện: True
----------------------------------------
Bàn cờ: [2 6 3 6 4 4 2 6]
h(board) = 8
Lời giải hợp lệ: False
Kiểm tra điều kiện: True
```

Bài 6: Các biến thể Hill Climbing

Task 1 — Steepest-ascent Hill Climbing

Ở mỗi vòng lặp, thuật toán duyệt qua toàn bộ các nước đi lân cận của bàn cờ hiện tại. Với mỗi nước đi, thuật toán tính số xung đột. Sau đó, thuật toán chọn nước đi có số xung đột nhỏ nhất.

Nếu không có nước đi nào giúp giảm số xung đột thì thuật toán dừng lại. Khi đó, bàn cờ có thể là lời giải hoặc cực tiểu cục bộ.

Mã code:

```
def steepest_hill_climbing(board):
    board = np.array(board, dtype=int).copy()
    n = len(board)
    steps = 0
    evaluations = 0
    history = [conflicts(board)]

    while True:
        current_h = conflicts(board)
        best_board = None
        best_h = current_h
        # Duyệt tất cả nước đi lân cận
        for col in range(n):
            for row in range(n):
                if row == board[col]:
                    continue

                neighbor = board.copy()
                neighbor[col] = row
                neighbor_h = conflicts(neighbor)
                evaluations += 1
                # Chỉ giữ nước đi tốt hơn trạng thái hiện tại
                if neighbor_h < best_h:
                    best_h = neighbor_h
                    best_board = neighbor.copy()

        # Không còn nước đi cải thiện
        if best_board is None:
            break

        board = best_board
        steps += 1
        history.append(best_h)

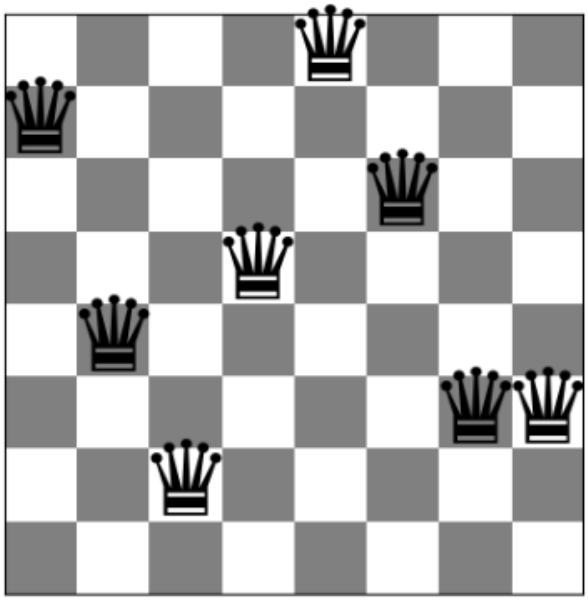
    final_h = conflicts(board)

    return {
        "board": board,
        "conflicts": final_h,
        "steps": steps,
        "evaluations": evaluations,
        "local_optimum": final_h > 0,
        "plateau": final_h > 0,
        "history": history
    }

# Thử nghiệm với bàn cờ 8 quân hậu
result1 = steepest_hill_climbing(random_board(8))
show_board(result1["board"])
print("Bàn cờ cuối:", result1["board"])
print("Số xung đột cuối:", result1["conflicts"])
print("Số bước:", result1["steps"])
print("Số lần đánh giá:", result1["evaluations"])
print("Mắc kẹt tại cực tiểu cục bộ:", result1["local_optimum"])
```

Kết quả:

```
Board with 1 conflicts.
```



```
Bàn cờ cuối: [1 4 6 3 0 2 5 5]
Số xung đột cuối: 1
Số bước: 4
Số lần đánh giá: 280
Mắc kẹt tại cực tiểu cục bộ: True
```

Giải thích: Hai vòng lặp for col và for row duyệt toàn bộ các nước đi có thể thực hiện. Mỗi cột có n - 1 nước đi hợp lệ vì không tính vị trí hàng hiện tại. Với bàn cờ n x n, mỗi vòng lặp đánh giá tối đa n(n - 1) nước đi. Thuật toán chọn trạng thái có số xung đột thấp nhất trong các trạng thái cải thiện nếu không tìm thấy trạng thái tốt hơn, thuật toán kết thúc.

Nhận xét: Thuật toán thường giảm xung đột khá nhanh ở những bước đầu. Tuy nhiên, thuật toán có thể dừng tại cực tiểu cục bộ khi vẫn còn xung đột. Biến evaluations giúp đo tổng số lần tính hàm mục tiêu trong quá trình tìm kiếm.

Task 2 — Inefficient Stochastic Hill Climbing

Thuật toán này cũng đánh giá tất cả các nước đi lân cận như Task 1. Điểm khác biệt là thay vì chọn nước đi cải thiện tốt nhất, thuật toán chọn ngẫu nhiên một nước đi trong tập hợp các nước đi cải thiện. Nếu không có nước đi cải thiện nào, thuật toán dừng lại.

Mã code:

```python
def inefficient_stochastic_hill_climbing(board):
    board = np.array(board, dtype=int).copy()
    n = len(board)
    steps = 0
    evaluations = 0
    history = [conflicts(board)]

    while True:
        current_h = conflicts(board)
        improving_moves = []
        # Đánh giá toàn bộ nước đi lân cận
        for col in range(n):
            for row in range(n):
                if row == board[col]:
                    continue

                neighbor = board.copy()
                neighbor[col] = row
                neighbor_h = conflicts(neighbor)
                evaluations += 1
                if neighbor_h < current_h:
                    improving_moves.append((neighbor, neighbor_h))

        # Không có nước đi cải thiện thì dừng
        if len(improving_moves) == 0:
            break

        # Chọn ngẫu nhiên một nước đi cải thiện
        selected = np.random.randint(len(improving_moves))
        board = improving_moves[selected][0]
        steps += 1
        history.append(conflicts(board))

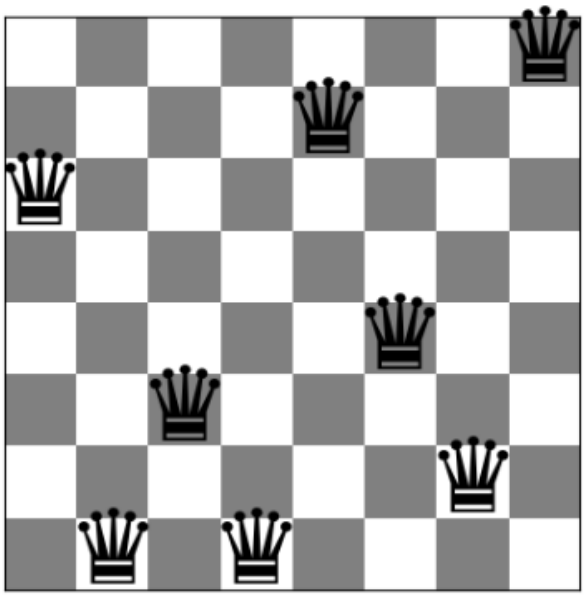
    final_h = conflicts(board)

    return {
        "board": board,
        "conflicts": final_h,
        "steps": steps,
        "evaluations": evaluations,
        "local_optimum": final_h > 0,
        "plateau": final_h > 0,
        "history": history
    }


# Thử nghiệm với bàn cờ 8 quân hậu
result2 = inefficient_stochastic_hill_climbing(random_board(8))
show_board(result2["board"])
print("Bàn cờ cuối:", result2["board"])
print("Số xung đột cuối:", result2["conflicts"])
print("Số bước:", result2["steps"])
print("Số lần đánh giá:", result2["evaluations"])
print("Mắc kẹt tại cực tiểu cục bộ:", result2["local_optimum"])
```

Kết quả:

```
Board with 2 conflicts.
```



```
Bàn cờ cuối: [2 7 5 7 1 4 6 0]
Số xung đột cuối: 2
Số bước: 3
Số lần đánh giá: 224
Mắc kẹt tại cực tiểu cục bộ: True
```

Giải thích: Thuật toán tạo danh sách improving_moves để lưu tất cả nước đi làm giảm số xung đột. Mỗi nước đi lân cận đều được đánh giá bằng hàm conflicts(). Nếu danh sách rỗng, thuật toán kết thúc còn nếu danh sách có phần tử, thuật toán chọn ngẫu nhiên một nước đi và chuyển sang trạng thái mới.

Nhận xét: Thuật toán có thể đi theo những hướng khác nhau giữa các lần chạy vì lựa chọn ngẫu nhiên. Tuy nhiên, do vẫn phải đánh giá tất cả các nước đi lân cận ở mỗi bước nên chi phí tính toán tương đối cao. So với Task 1, thuật toán không luôn chọn nước đi tốt nhất, vì vậy số bước và số xung đột cuối có thể khác nhau.

Task 3 — Efficient Stochastic Hill Climbing

Thuật toán này chỉ sinh một nước đi lân cận ngẫu nhiên tại mỗi lần thử, thay vì đánh giá toàn bộ các nước đi.

Quy trình thực hiện:

- Chọn ngẫu nhiên một cột và một hàng khác với hàng hiện tại.

- Tạo trạng thái mới và tính số xung đột.

- Nếu số xung đột giảm, chấp nhận nước đi và đặt lại bộ đếm số lần thử không thành công về 0.

- Nếu không cải thiện, giữ nguyên bàn cờ và tăng bộ đếm.

- Dừng khi tìm được lời giải hoặc số lần thử liên tiếp không cải thiện đạt ngưỡng x.

Vì không kiểm tra toàn bộ lân cận nên thuật toán không thể khẳng định chắc chắn đã đạt cực tiểu cục bộ. Ta sử dụng ngưỡng thử liên tiếp để phát hiện khả năng bị mắc kẹt.

Mã code:

```
def efficient_stochastic_hill_climbing(board, max_failures=None):
    board = np.array(board, dtype=int).copy()
    n = len(board)
    # Ngưỡng thử liên tiếp không cải thiện
    if max_failures is None:
        max_failures = 10 * n * (n - 1)
    steps = 0
    evaluations = 0
    failures = 0
    history = [conflicts(board)]

    while conflicts(board) > 0 and failures < max_failures:
        # Chọn ngẫu nhiên một cột
        col = np.random.randint(n)
        # Chọn một hàng khác hàng hiện tại
        possible_rows = [r for r in range(n) if r != board[col]]
        row = possible_rows[np.random.randint(len(possible_rows))]
        neighbor = board.copy()
        neighbor[col] = row
        neighbor_h = conflicts(neighbor)
        evaluations += 1
        # Chỉ chấp nhận nước đi cải thiện
        if neighbor_h < conflicts(board):
            board = neighbor
            steps += 1
            failures = 0
            history.append(neighbor_h)
        else:
            failures += 1

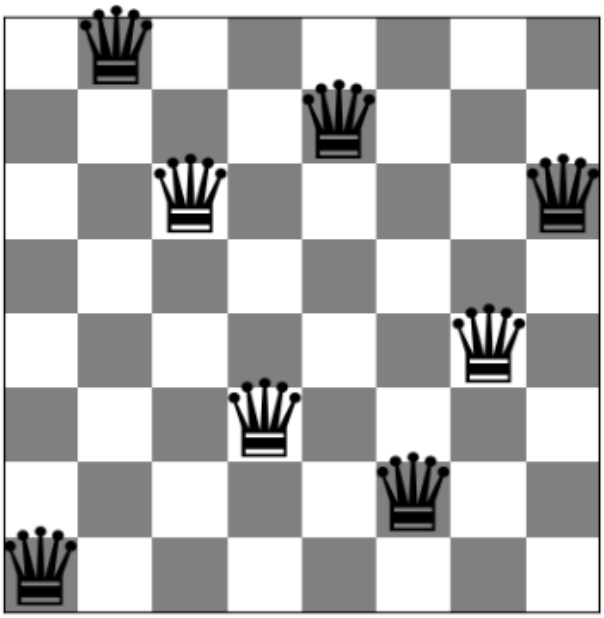
    final_h = conflicts(board)
    stopped_by_limit = failures >= max_failures and final_h > 0

    return {
        "board": board,
        "conflicts": final_h,
        "steps": steps,
        "evaluations": evaluations,
        "failures": failures,
        "local_optimum": stopped_by_limit,
        "plateau": stopped_by_limit,
        "history": history
    }
```

```python
# Thử nghiệm với bàn cờ 8 quân hậu
result3 = efficient_stochastic_hill_climbing(random_board(8))
show_board(result3["board"])
print("Bàn cờ cuối:", result3["board"])
print("Số xung đột cuối:", result3["conflicts"])
print("Số bước:", result3["steps"])
print("Số lần đánh giá:", result3["evaluations"])
print("Số lần thử không cải thiện liên tiếp:", result3["failures"])
print("Có khả năng mắc kẹt:", result3["local_optimum"])
```

Kết quả:

```
Board with 1 conflicts.
```



```
Bàn cờ cuối: [7 0 2 5 1 6 4 2]
Số xung đột cuối: 1
Số bước: 5
Số lần đánh giá: 766
Số lần thử không cải thiện liên tiếp: 560
Có khả năng mắc kẹt: True
```

Giải thích:

- max_failures = 10 * n * (n - 1) là ngưỡng thử liên tiếp không cải thiện được chọn để thực nghiệm. Đây là một giá trị quy ước, không phải ngưỡng tối ưu cho mọi kích thước bàn cờ.

- steps chỉ tăng khi thuật toán chấp nhận một nước đi cải thiện.

- evaluations tăng một lần cho mỗi nước đi được đánh giá.

- failures đếm số lần thử liên tiếp không cải thiện.

- local_optimum = True chỉ thể hiện thuật toán đã chạm ngưỡng dừng khi vẫn còn xung đột; điều này không chứng minh trạng thái là cực tiểu cục bộ thực sự.

Nhận xét: Thuật toán thường có số lần đánh giá thấp hơn hai thuật toán trước vì chỉ kiểm tra một nước đi ở mỗi lần thử. Tuy nhiên, thuật toán có thể bỏ lỡ những nước đi cải thiện khác và cần nhiều lần thử trước khi tìm được lời giải.

Ghi nhận và so sánh kết quả:

	Algorithm	n	Trials	Avg final conflicts	Avg steps	Avg evaluations	Success rate (%)	Stuck rate (%)
0	Steepest	4	100	0.64	1.46	29.52	41.0	59.0
1	Inefficient Stochastic	4	100	0.82	1.96	35.52	26.0	74.0
2	Efficient Stochastic	4	100	0.82	1.72	90.04	34.0	66.0
3	Steepest	8	100	1.22	3.42	247.52	12.0	88.0
4	Inefficient Stochastic	8	100	1.36	4.60	313.60	11.0	89.0
5	Efficient Stochastic	8	100	1.31	4.76	555.21	13.0	87.0

Chỉ số	**Ý nghĩa**
Avg final conflicts	Số xung đột trung bình khi thuật toán dừng
Avg steps	Số nước đi cải thiện được chấp nhận trung bình
Avg evaluations	Số lần đánh giá hàm mục tiêu trung bình
Success rate (%)	Tỷ lệ số lần tìm được bàn cờ có h(board) =
Stuck rate (%)	Tỷ lệ số lần thuật toán kết thúc tại trạng thái còn xung đột và được đánh dấu có khả năng mắc kẹt

Bài 7:

*** Task 4: chạy lại ba biến thể Hill Climbing từ các bàn cờ khởi tạo ngẫu nhiên, tối đa 100 lần khởi động lại; thống kê số lần thành công và chi phí tính toán.**

```python
# Task 4 – Thống kê số lần thành công của Random Restart
def summarize_random_restarts(
    name, algorithm, n=8, trials=100, max_restarts=100
):
    results = []

    # Repeat the complete random-restart experiment.
    for _ in range(trials):
        result = random_restart(
            algorithm,
            n=n,
            max_restarts=max_restarts
        )
        results.append(result)

    df = pd.DataFrame(results)

    success_count = int(df["Optimal"].sum())

    return {
        "Algorithm": name,
        "Board size": n,
        "Experiments": trials,
        "Successful experiments (h=0)": success_count,
        "Success rate (%)": success_count / trials * 100,
        "Average restarts": df["Restarts used"].mean(),
        "Average best h": df["Best conflicts"].mean(),
        "Best h observed": df["Best conflicts"].min(),
        "Average runtime (ms)": df["Runtime (ms)"].mean(),
        "Total runtime (s)": df["Runtime (ms)"].sum() / 1000
    }


restart_summary = []

for name, algorithm in hc_algorithms.items():
    summary = summarize_random_restarts(
        name,
        algorithm,
        n=8,
        trials=100,
        max_restarts=100
    )
    restart_summary.append(summary)

restart_table = pd.DataFrame(restart_summary)
display(restart_table.round(3))
```

	Algorithm	Board size	Experiments	Successful experiments (h=0)	Success rate (%)	Average restarts	Average best h	Best h observed	Average runtime (ms)	Total runtime (s)
0	Steepest asc. HC	8	100	100	100.0	7.41	0.0	0	40.899	4.090
1	Ineff. Stochastic HC	8	100	100	100.0	7.49	0.0	0	33.113	3.311
2	Efficient Stochastic HC	8	100	100	100.0	6.35	0.0	0	63.823	6.382

Từ bảng kết quả, cả ba thuật toán Hill Climbing đều đạt tỷ lệ thành công 100% sau 100 lần thử nghiệm trên bàn cờ kích thước 8. Chúng đều tìm được lời giải với giá trị heuristic bằng 0. Tuy nhiên, thời gian chạy và số lần khởi động lại có sự khác biệt.
Inefficient Stochastic HC có thời gian chạy trung bình thấp nhất, đạt 33,113 ms, trong khi Efficient Stochastic HC có số lần khởi động lại thấp nhất, đạt 6,35 lần, nhưng thời gian chạy trung bình cao nhất, đạt 63,823 ms. Steepest Ascent HC có kết quả nằm giữa hai thuật toán còn lại.

=> Vì vậy, nếu ưu tiên tốc độ thực thi trong thí nghiệm này, Inefficient Stochastic HC là lựa chọn tốt nhất; còn nếu ưu tiên giảm số lần khởi động lại, Efficient Stochastic HC có lợi thế hơn.

*** Task 5: cài đặt Simulated Annealing, thử ít nhất hai lịch làm nguội; nghiệm tệ hơn có thể được chấp nhận theo nhiệt độ và độ chênh hàm mục tiêu.**

```
# Task 5 – Thử các lịch làm nguội khác nhau
# Geometric, Linear và Logarithmic.
np.random.seed(2026)

n = 8
start_board = random_board(n)

T0 = 2.0
max_iter = 3000

cooling_schedules = [
    ("Geometric (alpha=0.995)", "geometric", 0.995),
    ("Linear", "linear", 0.995),
    ("Logarithmic", "logarithmic", 0.995)
]

schedule_results = []

plt.figure(figsize=(10, 5))

for name, schedule, alpha in cooling_schedules:
    best_board, current_history, best_history = (
        simulated_annealing(
            start_board.copy(),
            T0=T0,
            alpha=alpha,
            max_iter=max_iter,
            schedule=schedule
        )
    )

    best_h = conflicts(best_board)

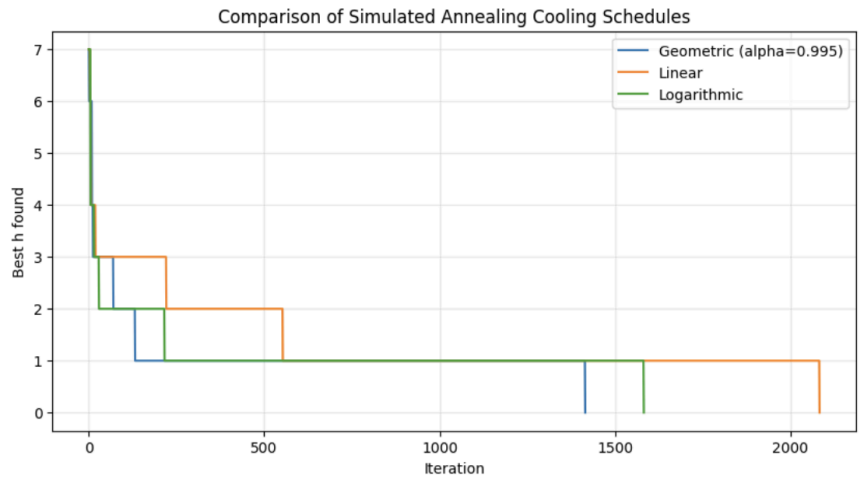
    schedule_results.append({
        "Cooling schedule": name,
        "Best h": best_h,
        "Success (h=0)": best_h == 0,
        "Iterations": len(current_history) - 1
    })

    plt.plot(best_history, label=name)

plt.xlabel("Iteration")
plt.ylabel("Best h found")
plt.title("Comparison of Simulated Annealing Cooling Schedules")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

display(pd.DataFrame(schedule_results))
```

...



	Cooling schedule	Best h	Success (h=0)	Iterations
0	Geometric (alpha=0.995)	0	True	1415
1	Linear	0	True	2083
2	Logarithmic	0	True	1582

Biểu đồ so sánh 3 lịch làm nguội (cooling schedules) của thuật toán Simulated Annealing (SA):

- Geometric ($\alpha = 0.995$): Lịch làm nguội hình học.
- Linear: Lịch làm nguội tuyến tính.
- Logarithmic: Lịch làm nguội logarithm.

Geometric — 1.415 vòng lặp: Đường màu xanh dương giảm khá nhanh xuống $h = 1$, sau đó tiếp tục tìm được $h = 0$ tại khoảng vòng lặp 1.415. Đây là lịch làm nguội có số vòng lặp thấp nhất trong ba phương pháp.

Linear — 2.083 vòng lặp: Đường màu cam duy trì $h = 3$ và $h = 2$ lâu hơn hai phương pháp còn lại, rồi giảm xuống $h = 1$ và cuối cùng đạt $h = 0$ tại khoảng vòng lặp 2.083. Phương pháp này cần nhiều vòng lặp nhất.

Logarithmic — 1.582 vòng lặp: Đường màu xanh lá giảm nhanh xuống $h = 1$, duy trì giá trị này trong một khoảng dài rồi đạt $h = 0$ tại khoảng vòng lặp 1.582. Kết quả tốt hơn Linear về số vòng lặp, nhưng chậm hơn Geometric.

Cả ba lịch làm nguội của Simulated Annealing đều tìm được lời giải với $h = 0$, cho thấy chúng đều thành công trong lần thử nghiệm này. Tuy nhiên, hiệu quả về số vòng lặp có sự khác biệt. Geometric với $\alpha = 0.995$ đạt lời giải sau 1.415 vòng lặp, là phương pháp hiệu quả nhất trong thử nghiệm. Logarithmic đứng thứ hai với 1.582 vòng lặp, trong khi Linear cần 2.083 vòng lặp, nhiều nhất trong ba phương pháp. Như vậy, Geometric là lựa chọn tốt nhất nếu xét theo số vòng lặp trong kết quả hiện tại.

Bài 8:

*** Task 6: chạy tối thiểu 100 lượt với các bàn cờ khởi tạo khác nhau tại n = 4 và n = 8; so sánh thời gian, h cuối, tỷ lệ thành công và trình bày kết quả bằng bảng/biểu đồ.**

```
def generate_unique_initial_boards(n, count=100):
    """Generate distinct board vectors for each experiment."""
    if count > n ** n:
        raise ValueError("Too many distinct initial boards requested.")

    boards = []
    seen = set()

    while len(boards) < count:
        board = np.random.randint(0, n, size=n)
        key = tuple(board.tolist())

        if key not in seen:
            seen.add(key)
            boards.append(board)

    return boards


def sa_benchmark_runner(start_board):
    best_board, current_history, best_history = (
        simulated_annealing(
            start_board,
            T0=2.0,
            alpha=0.995,
            max_iter=3000,
            schedule="geometric"
        )
    )

    return best_board, best_history


comparison_algorithms = {
    "Steepest asc. HC": steepest_hc,
    "Ineff. Stochastic HC": inefficient_stochastic_hc,
    "Efficient Stochastic HC": efficient_stochastic_hc,
    "Simulated Annealing": sa_benchmark_runner
}

np.random.seed(1234)

comparison_rows = []
```

```python
for n in [4, 8]:
    # Generate 100 different initial boards.
    initial_boards = [random_board(n) for _ in range(100)]

    for name, algorithm in comparison_algorithms.items():
        run_times = []
        final_conflict_counts = []

        for initial_board in initial_boards:
            start_time = time.perf_counter()

            final_board, history = algorithm(
                initial_board.copy()
            )

            elapsed = time.perf_counter() - start_time

            run_times.append(elapsed)
            final_conflict_counts.append(
                conflicts(final_board)
            )

        run_times = np.array(run_times)
        final_conflict_counts = np.array(
            final_conflict_counts
        )

        comparison_rows.append({
            "Algorithm": name,
            "Board size": n,
            "Avg. Run time (ms)": (
                np.mean(run_times) * 1000
            ),
            "Avg. number of conflicts": np.mean(
                final_conflict_counts
            ),
            "% Optimal": (
                np.mean(final_conflict_counts == 0) * 100
            )
        })

comparison_table = pd.DataFrame(comparison_rows)

display(
    comparison_table.round(3)
)
```

	Algorithm	Board size	Runs	Avg. runtime (ms)	Avg. final h	Best h observed	Successful runs (h=0)	Success rate (%)
0	Steepest asc. HC	4	100	0.421	0.60	0	45	45.0
1	Ineff. Stochastic HC	4	100	0.500	0.70	0	39	39.0
2	Efficient Stochastic HC	4	100	1.208	0.76	0	33	33.0
3	Simulated Annealing	4	100	1.186	0.00	0	100	100.0
4	Steepest asc. HC	8	100	2.915	1.27	0	21	21.0
5	Ineff. Stochastic HC	8	100	4.289	1.31	0	15	15.0
6	Efficient Stochastic HC	8	100	9.838	1.22	0	14	14.0
7	Simulated Annealing	8	100	18.791	0.03	0	97	97.0

	Algorithm	Board size	Avg. Run time (ms)	Avg. number of conflicts	% Optimal
0	Steepest asc. HC	4	0.242	0.54	53.0
1	Ineff. Stochastic HC	4	0.317	0.71	44.0
2	Efficient Stochastic HC	4	0.978	0.76	39.0
3	Simulated Annealing	4	1.128	0.00	100.0
4	Steepest asc. HC	8	3.065	1.32	11.0
5	Ineff. Stochastic HC	8	4.534	1.16	12.0
6	Efficient Stochastic HC	8	9.705	1.27	15.0
7	Simulated Annealing	8	21.058	0.04	96.0

- Bảng 1:

Trường hợp bàn cờ 4×4:

Steepest-ascent Hill Climbing (HC) nhanh nhất, chỉ mất trung bình 0.421 ms. Trong ba biến thể Hill Climbing, thuật toán này cũng có tỷ lệ thành công cao nhất, đạt 45%.

Inefficient Stochastic HC có tỷ lệ thành công 39%, thấp hơn Steepest-ascent HC.

Efficient Stochastic HC đạt tỷ lệ thành công 33% và thời gian chạy trung bình 1.208 ms. Trong kết quả này, thuật toán không thể hiện ưu thế về tốc độ hay tỷ lệ thành công so với hai biến thể HC còn lại.

Simulated Annealing (SA) tìm được lời giải trong cả 100 lần chạy, với \(h\) trung bình bằng 0. Tuy chậm hơn Steepest-ascent HC, SA có độ tin cậy cao hơn rõ rệt.

Kết luận cho bàn cờ 4 × 4: Nếu ưu tiên tốc độ, Steepest-ascent HC là lựa chọn tốt. Nếu ưu tiên khả năng tìm được lời giải, Simulated Annealing vượt trội.

Trường hợp bàn cờ 8 × 8:

Khi kích thước bàn cờ tăng từ 4 lên 8, thời gian chạy trung bình tăng ở cả bốn thuật toán.

Steepest-ascent HC vẫn nhanh nhất trong bốn thuật toán, mất 2.915 ms và đạt tỷ lệ thành công 21%. Tuy nhiên, khoảng 79% số lần chạy không kết thúc tại trạng thái có $h = 0$.

Inefficient Stochastic HC đạt tỷ lệ thành công 15%, thấp hơn Steepest-ascent HC.

Efficient Stochastic HC có giá trị h trung bình thấp nhất trong ba biến thể HC, ở mức 1.22. Tuy nhiên, nó mất 9.838 ms và chỉ đạt tỷ lệ thành công 14%. Điều này cho thấy giá trị heuristic trung bình thấp không nhất thiết đồng nghĩa với tỷ lệ tìm được lời giải cao nhất.

Simulated Annealing đạt tỷ lệ thành công 97%, tức 97/100 lần chạy tìm được lời giải. Đổi lại, thời gian chạy trung bình là 18.791 ms, khoảng 6,4 lần thời gian của Steepest-ascent HC.

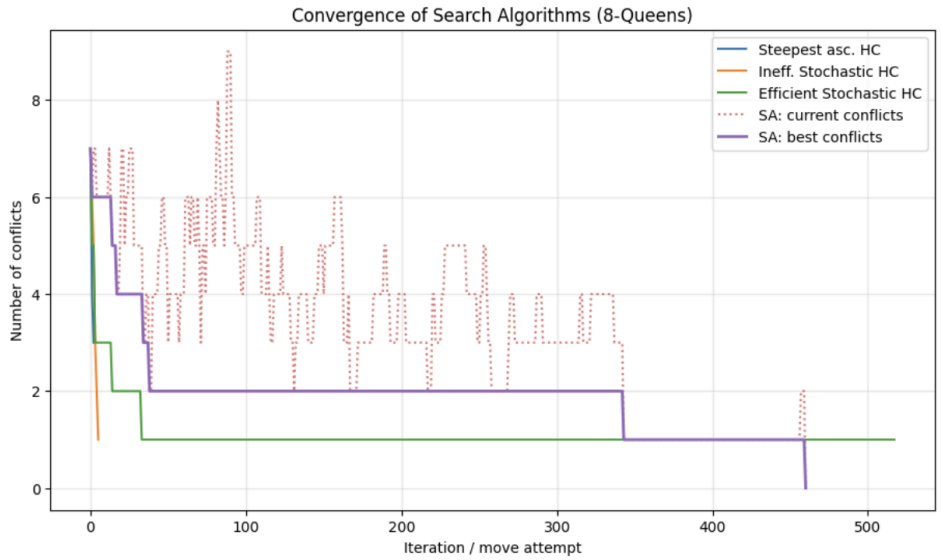
Kết luận cho bàn cờ 8 × 8: Simulated Annealing có ưu thế rõ rệt về khả năng tìm lời giải, còn Steepest-ascent HC phù hợp hơn khi cần tốc độ. Sự khác biệt giữa hai phương pháp trở nên đáng chú ý hơn khi kích thước bài toán tăng.

- Bảng 2:

Bàn cờ 4 x 4: Steepest-ascent HC có tốc độ cao, trong khi Simulated Annealing có tỷ lệ tối ưu cao nhất. SA đạt 100% tối ưu, còn ba biến thể HC đạt từ 39% đến 53%.

Bàn cờ 8 x 8: Khi bàn cờ tăng lên 8 × 8, tỷ lệ tối ưu của các thuật toán HC giảm xuống chỉ còn 11–15%, trong khi Simulated Annealing vẫn đạt 96%. Inefficient Stochastic HC có số xung đột trung bình thấp nhất trong ba thuật toán HC, ở mức 1.16, nhưng tỷ lệ tối ưu chỉ đạt 12%. Trong khi đó, Efficient Stochastic HC có số xung đột trung bình cao hơn một chút nhưng tỷ lệ tối ưu đạt 15%.

*** Vẽ đồ thị hội tụ h theo số vòng lặp cho một lượt chạy đại diện của mỗi thuật toán tại n = 8; nhận xét tốc độ cải thiện và plateau.**



- Steepest-ascent Hill Climbing (đường màu xanh dương): Đường biểu diễn của thuật toán này bị chồng lấp một phần với các đường khác nên khó xác định chính xác số xung đột cuối cùng chỉ từ hình ảnh. Nhìn chung, Hill Climbing chọn nước đi tốt nhất trong các trạng thái lân cận, giúp giảm xung đột nhanh khi tìm được hướng cải thiện.

- Inefficient Stochastic Hill Climbing (đường màu cam): Số xung đột giảm nhanh từ khoảng 7 xuống còn 1 chỉ sau một vài bước được chấp nhận. Điều này cho thấy thuật toán có thể tìm được nước đi cải thiện hiệu quả nhờ lựa chọn ngẫu nhiên trong tập các nước đi tốt hơn.

- Efficient Stochastic Hill Climbing (đường màu xanh lá):
  + Số xung đột giảm nhanh từ khoảng 7 xuống 3, sau đó xuống 2 và cuối cùng còn 1. Sau đó, đường biểu diễn nằm ngang trong một khoảng dài dù thuật toán tiếp tục thử các nước đi.
  + Nguyên nhân là thuật toán chỉ xem xét một nước đi ngẫu nhiên ở mỗi lần thử. Khi nước đi không cải thiện, trạng thái hiện tại được giữ nguyên. Vì vậy, thuật toán có thể mất nhiều lần thử mà không cải thiện được kết quả.

- Simulated Annealing (SA):
  + Đường chấm màu đỏ biểu diễn số xung đột của trạng thái hiện tại. Giá trị dao động mạnh, có lúc tăng lên khoảng 9 rồi giảm xuống. Đây là hành vi bình thường vì thuật toán có thể chấp nhận những nước đi làm tăng xung đột để thoát khỏi cực tiểu cục bộ.
  + Đường màu tím biểu diễn số xung đột tốt nhất đã tìm được. Đường này chỉ giảm hoặc giữ nguyên: từ khoảng 7 xuống 2, duy trì khá lâu, sau đó giảm xuống 1 và cuối cùng đạt 0 xung đột ở khoảng bước 460. Như vậy, Simulated Annealing đã tìm được nghiệm tối ưu trong lần chạy được minh họa, mặc dù trạng thái hiện tại vẫn có thể chứa xung đột.

*** Vẽ biểu đồ log-log thời gian chạy theo n ∈ {4, 8, 12, 16, 20} cho ít nhất hai thuật toán; thảo luận độ phức tạp thực nghiệm và sự phụ thuộc vào cài đặt.**

```
# Code and description go here
plt.figure(figsize=(9, 6))

for name in scaling_algorithms:
    subset = scaling_table[
        scaling_table["Algorithm"] == name
    ].sort_values("Board size")

    x = subset["Board size"].to_numpy(dtype=float)
    y = subset["Avg. runtime (s)"].to_numpy(dtype=float)

    plt.loglog(
        x, y,
        marker="o",
        label=name
    )

    # Fit log(time) = p * log(n) + intercept.
    p, intercept = np.polyfit(
        np.log(x), np.log(y), 1
    )

    print(
        f"{name}: estimated exponent p = {p:.2f}; "
        f"empirical runtime approximately proportional "
        f"to n^{p:.2f} over the tested range."
    )

plt.xlabel("Board size n (log scale)")
plt.ylabel("Average runtime in seconds (log scale)")
plt.title("Scalability of Hill Climbing Algorithms")
plt.legend()
plt.grid(True, which="both", alpha=0.3)
plt.show()
```

```
# Problem Size Scalability
# Code and description go here
scaling_algorithms = {
    "Steepest asc. HC": steepest_hc,
    "Efficient Stochastic HC": efficient_stochastic_hc
}

board_sizes = [4, 8, 12, 16, 20]
repeats = 30

scaling_rows = []

for n in board_sizes:
    # Use independent random starting boards.
    initial_boards = [
        random_board(n) for _ in range(repeats)
    ]

    for name, algorithm in scaling_algorithms.items():
        run_times = []

        for initial_board in initial_boards:
            start_time = time.perf_counter()

            final_board, history = algorithm(
                initial_board.copy()
            )

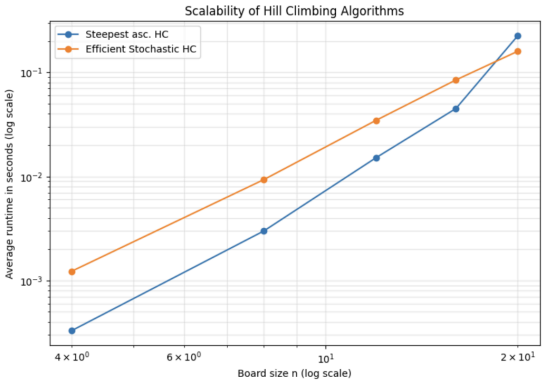
            elapsed = time.perf_counter() - start_time
            run_times.append(elapsed)

        scaling_rows.append({
            "Algorithm": name,
            "Board size": n,
            "Avg. runtime (s)": np.mean(run_times)
        })

scaling_table = pd.DataFrame(scaling_rows)
display(scaling_table.round(6))
```

	Algorithm	Board size	Avg. runtime (s)
0	Steepest asc. HC	4	0.000330
1	Efficient Stochastic HC	4	0.001229
2	Steepest asc. HC	8	0.002995
3	Efficient Stochastic HC	8	0.009331
4	Steepest asc. HC	12	0.015183
5	Efficient Stochastic HC	12	0.034704
6	Steepest asc. HC	16	0.044998
7	Efficient Stochastic HC	16	0.084683
8	Steepest asc. HC	20	0.225287
9	Efficient Stochastic HC	20	0.159788

```
Steepest asc. HC: estimated exponent p = 3.91; empirical runtime approximately proportional to n^3.91 over the tested range.
Efficient Stochastic HC: estimated exponent p = 3.05; empirical runtime approximately proportional to n^3.05 over the tested range.
```



Qua bảng thời gian chạy và biểu đồ log-log, có thể thấy thời gian thực hiện của cả Steepest-ascent Hill Climbing và Efficient Stochastic Hill Climbing đều tăng khi kích thước bàn cờ tăng từ 4 lên 20. Trong thí nghiệm này, Steepest-ascent HC chạy nhanh hơn ở các kích thước từ 4 đến 16, nhưng Efficient Stochastic HC nhanh hơn ở kích thước 20.

Kết quả hồi quy log-log cho số mũ thực nghiệm $p = 3.91$ đối với Steepest-ascent HC và $p = 3.05$ đối với Efficient Stochastic HC. Điều này gợi ý rằng thời gian chạy của Efficient

Stochastic HC tăng chậm hơn trên phạm vi dữ liệu đã khảo sát. Nguyên nhân có thể là thuật toán chỉ kiểm tra một nước đi tại mỗi lần thử, trong khi Steepest-ascent HC phải đánh giá toàn bộ các trạng thái lân cận.

Tuy nhiên, thời gian chạy của Efficient Stochastic HC còn phụ thuộc số lần thử không cải thiện và điều kiện dừng. Các số mũ trên chỉ phản ánh xu hướng thực nghiệm trong phạm vi $n = 4, 8, 12, 16, 20$, không phải độ phức tạp tiệm cận được chứng minh. Nhìn chung, Efficient Stochastic HC có triển vọng giảm chi phí đánh giá trạng thái khi kích thước bài toán lớn, nhưng cần thêm các lần thử và kích thước lớn hơn để xác nhận xu hướng này.

II. Bảng ghi kết quả thí nghiệm

Bài toán	Kích thước	Thuật toán	Số màu / lượt	Thành công (%)	Xung đột / COUNT	Thời gian (s)
Tô màu	n = 10	Backtracking	4	100%	COUNT = 11	0,000208
Tô màu	n = 10	MRV + Backtracking	4	100%	COUNT = 11	0,000474
Tô màu	n = 10	Forward Checking	3 / 4	53,3%*	COUNT TB: 9–34	0,00013–0,00066
Tô màu	n thay đổi	Min-Conflicts	3 / 4	53,3%*	Số bước TB: 5,6–5.000	0,00041–0,24405
n-Queens	n = 4	Steepest HC	≥100	53%	0.54	0.000242
n-Queens	n = 8	Stochastic HC	≥100	12%	1.16	0.004534
n-Queens	n = 8	Random Restart	≥100	100%	0.00	0.040899
n-Queens	n = 8	Simulated Annealing	≥100	100%	0.04	0.021058

III. Câu hỏi thảo luận

1. Khác nhau giữa complete() và consistent()? Một phép gán đầy đủ có luôn là nghiệm CSP không?

complete() kiểm tra xem tất cả các biến đã được gán giá trị hay chưa. consistent() kiểm tra xem các giá trị đã gán có vi phạm các ràng buộc của CSP hay không.

Một phép gán đầy đủ chưa chắc là nghiệm CSP, vì dù tất cả biến đã có giá trị nhưng vẫn có thể vi phạm ràng buộc. Muốn là nghiệm CSP, phép gán phải vừa đầy đủ, vừa nhất quán.

2. MRV và Forward Checking tác động thế nào đến cây Backtracking? Khi nào kiểm tra miền làm tăng chi phí?

MRV chọn biến chưa gán có ít giá trị hợp lệ nhất để gán trước, giúp ưu tiên những biến bị hạn chế nhiều và có thể phát hiện sớm trường hợp không có nghiệm. Forward Checking cập nhật miền giá trị của các biến chưa gán sau mỗi lần gán, loại bỏ những giá trị không còn phù hợp với ràng buộc.

Hai kỹ thuật này có thể giảm số nhánh cần tìm kiếm, nhưng cũng làm tăng chi phí kiểm tra và cập nhật miền. Khi miền có nhiều giá trị, ràng buộc ít hoặc việc kiểm tra không loại bỏ được nhiều giá trị, chi phí bổ sung có thể lớn hơn lợi ích mang lại.

3. Vì sao tô màu bằng 3 màu có thể thất bại dù dùng thuật toán đúng? Làm sao phân biệt hết giới hạn tính toán và chứng minh vô nghiệm?

Bài toán tô màu bằng 3 màu có thể không tìm được nghiệm vì đồ thị không thể tô màu hợp lệ chỉ bằng 3 màu, hoặc thuật toán chưa tìm được lời giải trong giới hạn thời gian hay số bước cho phép. Ví dụ, đồ thị đầy đủ có 4 đỉnh cần ít nhất 4 màu nên không thể tô hợp lệ bằng 3 màu.

Nếu thuật toán tìm kiếm đầy đủ đã duyệt hết các khả năng mà không tìm được nghiệm thì có thể kết luận bài toán vô nghiệm với số màu đã cho. Ngược lại, nếu chương trình dừng do hết thời gian, đạt giới hạn số bước hoặc bị giới hạn tài nguyên, chỉ có thể kết luận chưa tìm được nghiệm, chứ chưa thể khẳng định bài toán vô nghiệm.

4. Hill Climbing gap local optimum hoặc plateau ra sao? Random Restart và Simulated Annealing xử lý theo cách nào?

Hill Climbing có thể bị mắc kẹt tại local optimum (cực tiểu cục bộ) khi không còn trạng thái lân cận nào tốt hơn, dù chưa tìm được lời giải tối ưu. Với plateau (vùng bằng phẳng), các trạng thái lân cận có cùng giá trị hàm mục tiêu khiến thuật toán khó tìm hướng cải thiện.

- Random Restart Hill Climbing: Khởi động lại thuật toán với trạng thái ban đầu ngẫu nhiên mới, giúp tăng khả năng tìm được lời giải khác.
- Simulated Annealing: Thỉnh thoảng chấp nhận cả nước đi xấu hơn với một xác suất nhất định, giúp thuật toán có cơ hội thoát khỏi cực tiểu cục bộ. Xác suất này giảm dần theo nhiệt độ.

5. So sánh Steepest, Inefficient Stochastic và Efficient Stochastic về số lần đánh giá các láng giềng.

- Steepest Hill Climbing: Đánh giá toàn bộ các trạng thái lân cận ở mỗi bước để chọn nước đi cải thiện tốt nhất.
- Inefficient Stochastic Hill Climbing: Cũng đánh giá toàn bộ các trạng thái lân cận, sau đó chọn ngẫu nhiên một nước đi trong số các nước cải thiện.
- Efficient Stochastic Hill Climbing: Chỉ đánh giá lần lượt một trạng thái lân cận ngẫu nhiên tại mỗi lần thử và chấp nhận nếu trạng thái đó cải thiện.

Vì vậy, hai thuật toán đầu thường cần nhiều lần đánh giá hơn mỗi bước, còn Efficient Stochastic tiết kiệm số lần đánh giá hơn nhưng có thể cần nhiều lần thử mới tìm được nước đi cải thiện.

6. Vì sao cần nhiều lần chạy và cố định điều kiện thí nghiệm khi đánh giá thuật toán ngẫu nhiên?

Các thuật toán ngẫu nhiên có thể cho kết quả khác nhau giữa các lần chạy do trạng thái khởi đầu và cách chọn nước đi ngẫu nhiên khác nhau. Vì vậy, cần chạy nhiều lần để đánh giá khách quan hơn về tỷ lệ thành công, số bước và số lần đánh giá trung bình.

Đồng thời, cần cố định các điều kiện thí nghiệm như kích thước bàn cờ, số lần chạy, điều kiện dừng và cách tính hàm mục tiêu để đảm bảo các thuật toán được so sánh công bằng và kết quả có thể tái lập.